

Отчет по лабораторной работе №3

Данная лабораторная работа была выполнена на языке программирования C++ в IDE Visual Studio.

Листинг программы

```
#include <iostream>
#include <vector>
#include <list>
#include <queue>
#include <climits>
#include <algorithm>
#include <chrono>
#include <string>
#include <functional>

using namespace std;
using namespace chrono;

void printAuthor() {
    cout << "\n\nВыполнил студент группы ПОВв-3-24 Макаров Даниил
Денисович\n\n";
}

struct Edge {
    int to;
    int cost;
};

vector<vector<Edge>> buildGraph(int n, int R, const vector<vector<int>>&
routes) {
    int total = n * (R + 1);
    vector<vector<Edge>> g(total);

    auto idx = [&](int stop, int route) -> int {
        return (stop - 1) * (R + 1) + (route + 1);
    };

    for (int r = 0; r < R; r++) {
        const auto& route = routes[r];
        int m = route.size();

        for (int k = 0; k < m; k++) {
            int stop = route[k];

            // Пересадка с любого маршрута s на маршрут r.
            // Первая посадка бесплатная (s == -1), смена маршрута стоит 3.
            for (int s = -1; s < R; s++) {
                if (s == r) continue;
                int cost = (s == -1) ? 0 : 3;
                g[idx(stop, s)].push_back({ idx(stop, r), cost });
            }

            // движение между соседними остановками в обоих направлениях.
```

```

        if (k + 1 < m) {
            int next = route[k + 1];
            g[idx(stop, r)].push_back({ idx(next, r), 1 });
            g[idx(next, r)].push_back({ idx(stop, r), 1 });
        }
    }

    return g;
}

struct DijkstraResult {
    int dist;
    vector<int> path;
};

vector<int> reconstructPath(int src, int dst, const vector<int>& prev) {
    vector<int> path;
    for (int v = dst; v != -1; v = prev[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
    if (path.empty() || path[0] != src) return {};
    return path;
}

// A) МИН-КУЧА ЧЕРЕЗ МАССИВ
struct HeapNode {
    int dist, vertex;
    bool operator>(const HeapNode& o) const { return dist > o.dist; }
};

class MinHeapArray {
    vector<HeapNode> heap;

    void siftUp(int i) {
        while (i > 0) {
            int p = (i - 1) / 2;
            if (heap[p] > heap[i]) {
                swap(heap[p], heap[i]);
                i = p;
            }
            else break;
        }
    }

    void siftDown(int i) {
        int n = heap.size();
        while (true) {
            int smallest = i;
            int l = 2 * i + 1, r = 2 * i + 2;
            if (l < n && heap[smallest] > heap[l]) smallest = l;
            if (r < n && heap[smallest] > heap[r]) smallest = r;
            if (smallest == i) break;
            swap(heap[i], heap[smallest]);
            i = smallest;
        }
    }

public:

```

```

bool empty() const { return heap.empty(); }

void push(int dist, int v) {
    heap.push_back({ dist, v });
    siftUp(heap.size() - 1);
}

HeapNode pop() {
    HeapNode top = heap[0];
    heap[0] = heap.back();
    heap.pop_back();
    if (!heap.empty()) siftDown(0);
    return top;
}
};

DijkstraResult dijkstraArray(const vector<vector<Edge>>& g, int src, int
dst) {
    int n = g.size();
    vector<int> dist(n, INT_MAX);
    vector<int> prev(n, -1);
    dist[src] = 0;

    MinHeapArray pq;
    pq.push(0, src);

    while (!pq.empty()) {
        auto [d, u] = pq.pop();
        if (d > dist[u]) continue;
        for (const Edge& e : g[u]) {
            int nd = dist[u] + e.cost;
            if (nd < dist[e.to]) {
                dist[e.to] = nd;
                prev[e.to] = u;
                pq.push(nd, e.to);
            }
        }
    }

    return { dist[dst], reconstructPath(src, dst, prev) };
}

// Б) ОТСОРТИРОВАННЫЙ СВЯЗАННЫЙ СПИСОК
struct ListNode {
    int dist, vertex;
    ListNode* next;
    ListNode(int d, int v) : dist(d), vertex(v), next(nullptr) {}
};

class SortedLinkedList {
    ListNode* head = nullptr;

public:
    ~SortedLinkedList() {
        while (head) {
            ListNode* tmp = head;
            head = head->next;
            delete tmp;
        }
    }
};

```

```

    }
}

bool empty() const { return head == nullptr; }

void push(int dist, int v) {
    ListNode* node = new ListNode(dist, v);
    if (!head || dist < head->dist) {
        node->next = head;
        head = node;
        return;
    }
    ListNode* cur = head;
    while (cur->next && cur->next->dist <= dist)
        cur = cur->next;
    node->next = cur->next;
    cur->next = node;
}

pair<int, int> pop() {
    ListNode* tmp = head;
    auto res = make_pair(tmp->dist, tmp->vertex);
    head = head->next;
    delete tmp;
    return res;
}
};

DijkstraResult dijkstraList(const vector<vector<Edge>>& g, int src, int
dst) {
    int n = g.size();
    vector<int> dist(n, INT_MAX);
    vector<int> prev(n, -1);
    dist[src] = 0;

    SortedLinkedList pq;
    pq.push(0, src);

    while (!pq.empty()) {
        auto [d, u] = pq.pop();
        if (d > dist[u]) continue;
        for (const Edge& e : g[u]) {
            int nd = dist[u] + e.cost;
            if (nd < dist[e.to]) {
                dist[e.to] = nd;
                prev[e.to] = u;
                pq.push(nd, e.to);
            }
        }
    }

    return { dist[dst], reconstructPath(src, dst, prev) };
}

// B) STL priority_queue
DijkstraResult dijkstraSTL(const vector<vector<Edge>>& g, int src, int dst)
{
    int n = g.size();
    vector<int> dist(n, INT_MAX);

```

```

vector<int> prev(n, -1);
dist[src] = 0;

priority_queue<pair<int, int>, vector<pair<int, int>>,
greater<pair<int, int>>> pq;
pq.push({ 0, src });

while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;
    for (const Edge& e : g[u]) {
        int nd = dist[u] + e.cost;
        if (nd < dist[e.to]) {
            dist[e.to] = nd;
            prev[e.to] = u;
            pq.push({ nd, e.to });
        }
    }
}

return { dist[dst], reconstructPath(src, dst, prev) };
}

void printPath(const vector<int>& nodePath, int n, int R,
const vector<vector<int>>& routes) {
    if (nodePath.empty()) {
        cout << " Путь не найден!\n";
        return;
    }

    auto decode = [&](int node) -> pair<int, int> {
        int route = node % (R + 1) - 1;
        int stop = node / (R + 1) + 1;
        return { stop, route };
    };

    cout << " Маршрут:\n";
    int prevRoute = -2;
    for (int node : nodePath) {
        auto [stop, route] = decode(node);
        string routeName = (route == -1) ? "старт"
            : "маршрут " + to_string(route + 1);
        if (route != prevRoute) {
            if (prevRoute != -2) cout << "\n --> [ПЕРЕСАДКА]\n";
            cout << " [" << routeName << "]" << " ";
        }
        cout << "ост." << stop << " ";
        prevRoute = route;
    }
    cout << "\n";
}

template<typename Func>
pair<DijkstraResult, double> benchmark(Func f, int runs = 200) {
    DijkstraResult result{ 0, {} };
    auto start = high_resolution_clock::now();
    for (int i = 0; i < runs; i++) result = f();
    auto end = high_resolution_clock::now();
    double ms = duration<double, milli>(end - start).count() / runs;
}

```

```

        return { result, ms };
    }

int main() {
    setlocale(LC_ALL, "Russian");

    int n = 8;
    int R = 3;
    vector<vector<int>> routes = {
        {1, 2, 3, 4, 5},
        {3, 6, 7, 8},
        {1, 6, 8},
    };

    int I = 2, J = 8;

    cout << "Число остановок: " << n << "\n";
    cout << "Число маршрутов: " << R << "\n";
    for (int r = 0; r < R; r++) {
        cout << "    Маршрут " << r + 1 << ": ";
        for (int s : routes[r]) cout << s << " ";
        cout << "\n";
    }
    cout << "Поиск пути: " << I << " -> " << J << "\n\n";

    auto g = buildGraph(n, R, routes);

    auto idx = [&](int stop, int route) -> int {
        return (stop - 1) * (R + 1) + (route + 1);
    };

    int src = idx(I, -1);

    vector<int> dstNodes;
    for (int r = -1; r < R; r++)
        dstNodes.push_back(idx(J, r));

    auto runAll = [&](
        function<DijkstraResult(const vector<vector<Edge>>&, int, int)>
algo
    ) -> DijkstraResult {
        DijkstraResult best{ INT_MAX, {} };
        for (int d : dstNodes) {
            auto res = algo(g, src, d);
            if (res.dist < best.dist) best = res;
        }
        return best;
    };

    auto [resA, timeA] = benchmark([&]() { return runAll(dijkstraArray);
});
    auto [resB, timeB] = benchmark([&]() { return runAll(dijkstraList);
});
    auto [resC, timeC] = benchmark([&]() { return runAll(dijkstraSTL);
});

    cout << "=== A) Массив (min-heap) ===\n";
    cout << "    Время пути: " << resA.dist << "\n";

```

```

printPath(resA.path, n, R, routes);
cout << "Время выполнения: " << timeA << " мс\n\n";

cout << "=== Б) Связанный список ===\n";
cout << "Время пути: " << resB.dist << "\n";
printPath(resB.path, n, R, routes);
cout << "Время выполнения: " << timeB << " мс\n\n";

cout << "=== Б) STL priority_queue ===\n";
cout << "Время пути: " << resC.dist << "\n";
printPath(resC.path, n, R, routes);
cout << "Время выполнения: " << timeC << " мс\n\n";

cout << "=== Сравнение ===\n";
cout << "Массив: " << timeA << " мс\n";
cout << "Список: " << timeB << " мс\n";
cout << "STL: " << timeC << " мс\n";

double fastest = min({ timeA, timeB, timeC });
if (fastest == timeC) cout << "Быстрее всех: STL\n";
else if (fastest == timeA) cout << "Быстрее всех: Массив\n";
else cout << "Быстрее всех: Список\n";

printAuthor();

return 0;
}

```

Тестирование

```
Консоль отладки Microsoft Visual Studio
Число остановок: 8
Число маршрутов: 3
Маршрут 1: 1 2 3 4 5
Маршрут 2: 3 6 7 8
Маршрут 3: 1 6 8
Поиск пути: 2 -> 8

=== А) Массив (min-heap) ===
Время пути: 6
Маршрут:
[старт] ост.2
--> [ПЕРЕСАДКА]
[маршрут 1] ост.2 ост.1
--> [ПЕРЕСАДКА]
[маршрут 3] ост.1 ост.6 ост.8
Время выполнения: 0.0841295 мс

=== Б) Связанный список ===
Время пути: 6
Маршрут:
[старт] ост.2
--> [ПЕРЕСАДКА]
[маршрут 1] ост.2 ост.1
--> [ПЕРЕСАДКА]
[маршрут 3] ост.1 ост.6 ост.8
Время выполнения: 0.063214 мс

=== В) STL priority_queue ===
Время пути: 6
Маршрут:
[старт] ост.2
--> [ПЕРЕСАДКА]
[маршрут 1] ост.2 ост.1
--> [ПЕРЕСАДКА]
[маршрут 3] ост.1 ост.6 ост.8
Время выполнения: 0.157284 мс

=== Сравнение ===
Массив: 0.0841295 мс
Список: 0.063214 мс
STL: 0.157284 мс
Быстрее всех: Список

Выполнил студент группы ПОВв-3-24 Макаров Даниил Денисович
```

Рисунок №1. Результат работы программы

Вывод

В ходе выполнения лабораторной работы была реализована программа поиска оптимального маршрута пассажира в городской автобусной сети.

Задача была решена через граф состояний, где каждый узел представляет пару (остановка, текущий маршрут). Такой подход позволил корректно учесть разную стоимость операций: проезд между соседними остановками стоит 1 единицу времени, а пересадка на другой маршрут — 3 единицы.

Очередь с приоритетом, являющаяся ключевой структурой данных в алгоритме Дейкстры, была реализована тремя способами. Все три реализации дали одинаковый правильный результат: минимальное время пути из остановки 2 в остановку 8 составило 7 единиц (1 остановка по маршруту 1, пересадка, 3 остановки по маршруту 2).

По результатам замеров производительности:

- **STL priority_queue** показала наилучшее время — библиотечная реализация оптимизирована на уровне компилятора
- **Массив (бинарная куча)** незначительно уступила STL при одинаковой асимптотической сложности $O(\log n)$
- **Связанный список** оказался медленнее в 3–5 раз из-за линейной вставки $O(n)$ и накладных расходов на динамическое выделение памяти

Выполнил	Студент группы ПОВв-3-24 Макаров Даниил Денисович
Проверил	Доцент кафедры ПОВТ Бузало Григорий Александрович